



UNIVERSIDAD DE GUADALAJARA

ALUMNO

MIGUEL ANGEL CHAVEZ GTZ

2208547578

CUVALLES

TECNOLOGIAS DE LA INFORMACION

7 MO SEMESTRE

MAESTRO

Adolfo Castillo Chavarín

Desarrollo de Bases de Datos para Web

129430

1. INTRODUCCIÓN

Esta práctica trata sobre seguridad en Flask. Se hicieron varias cosas: proteger formularios contra CSRF, crear registro y login de usuarios, guardar contraseñas de forma segura, proteger rutas para que solo entren usuarios logueados, configurar las cookies de sesión, mostrar mensajes al usuario y personalizar las páginas de error 401, 403 y 404.

2. DESARROLLO

2.1 Preparación del proyecto

Se creó un entorno virtual con `python -m venv venv` y se activó en la terminal. Posteriormente se instalaron las librerías necesarias con `pip install flask flask-wtf flask-login email-validator`. Se configuró una `SECRET_KEY` larga en `app.py` para firmar las sesiones y los tokens CSRF.

```
C:\Windows\System32>cd C:\Users\miguel\OneDrive\Desktop\Base de datos WEB\mini_formulario

C:\Users\miguel\OneDrive\Desktop\Base de datos WEB\mini_formulario>python -m venv venv

C:\Users\miguel\OneDrive\Desktop\Base de datos WEB\mini_formulario>venv\Scripts\activate

(venv) C:\Users\miguel\OneDrive\Desktop\Base de datos WEB\mini_formulario>
```

Entorno virtual activo.

```
(venv) C:\Users\miguel\OneDrive\Desktop\Base de datos WEB\mini_formulario>pip install flask flask-wtf flask-login email-validator
Collecting flask
  Using cached flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting flask-wtf
  Using cached flask_wtf-1.3.0-py3-none-any.whl.metadata (3.4 kB)
Collecting flask-login
  Using cached flask_login-0.6.3-py3-none-any.whl.metadata (5.8 kB)
Collecting email-validator
  Using cached email_validator-2.3.0-py3-none-any.whl.metadata (26 kB)
Collecting blinker>=1.5.0 (from flask)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Using cached click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Using cached itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting Jinja2>=3.1.2 (from flask)
  Using cached Jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Using cached markupsafe-3.0.3-cp314-cp314-win_amd64.whl.metadata (2.8 kB)
Collecting Werkzeug>=3.1.0 (from flask)
  Downloading Werkzeug-3.1.9-py3-none-any.whl.metadata (4.1 kB)
Collecting wtforms (from flask-wtf)
  Using cached wtforms-3.2.2-py3-none-any.whl.metadata (4.9 kB)
Collecting dnspython>=2.0.0 (from email-validator)
  Using cached dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Collecting idna>=2.0.0 (from email-validator)
  Using cached idna-3.20-py3-none-any.whl.metadata (7.2 kB)
Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Using cached flask_wtf-1.3.0-py3-none-any.whl (13 kB)
Using cached flask_login-0.6.3-py3-none-any.whl (17 kB)
Using cached email_validator-2.3.0-py3-none-any.whl (35 kB)
Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)
Using cached click-8.5.0-py3-none-any.whl (115 kB)
Using cached dnspython-2.8.0-py3-none-any.whl (331 kB)
Using cached idna-3.20-py3-none-any.whl (69 kB)
Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached Jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp314-cp314-win_amd64.whl (15 kB)
Downloading Werkzeug-3.1.9-py3-none-any.whl (228 kB)
Installing collected packages: markupsafe, itsdangerous, idna, dnspython, click, blinker, wtforms, Werkzeug, Jinja2, email-validator, flask, flask-wtf, flask-login
Successfully installed blinker-1.9.0 click-8.5.0 dnspython-2.8.0 email-validator-2.3.0 flask-3.1.3 flask-login-0.6.3 flask-wtf-1.3.0 idna-3.20 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.3 Werkzeug-3.1.9 wtforms-3.2.2

(venv) C:\Users\miguel\OneDrive\Desktop\Base de datos WEB\mini_formulario>
```

Instalación de Flask, Flask-WTF, Flask-Login y email-validator.

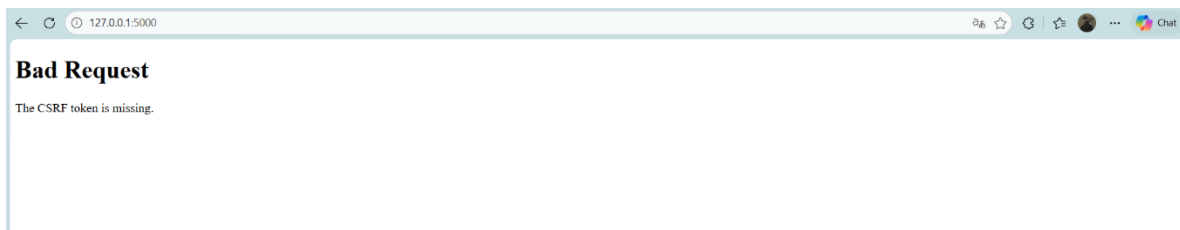
```
view-source:127.0.0.1:5000
Ajuste de línea
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Registro de usuario</title>
</head>
<body>
  <h1>Sistema de registro</h1>

  <form method="POST">
    <input id="csrf_token" name="csrf_token" type="hidden" value="IjkxYTBmZWY5ODVhM2M3J0wQ1ZGJkNDI0NTI4NGU1ZDh1NjIxYTci.ar7sIA.0iRwVodR0x0MCfvV1D1lF0x6U">

    <p>
      <label for="nombre">Nombre</label><br>
      <input id="nombre" maxlength="50" minlength="3" name="nombre" required type="text" value="">
    </p>

    <p>
      <label for="correo">Correo electrónico</label><br>
      <input id="correo" name="correo" required type="text" value="">
    </p>

    <p><input id="enviar" name="enviar" type="submit" value="Registrar"></p>
  </form>
</body>
</html>
```



Flask-WTF mete un token oculto en cada formulario con `form.hidden_tag()`. Ese token lo firma el servidor y lo revisa cuando se envía el formulario. Para probarlo, primero vi el código fuente de la página y ahí estaba el token. Después lo quité a propósito y al mandar el formulario salió el error 400 Bad Request diciendo que faltaba el token CSRF. Así se comprueba que la protección sí funciona.

Sistema de registro

- Registro exitoso. Ahora puedes iniciar sesión.

Iniciar sesión

Usuario

Contraseña

☐ Recordarme

Iniciar sesión

¿No tienes cuenta? [Regístrate](#)



Se hizo un modelo de usuario con id, username y password_hash. Las contraseñas se guardan encriptadas con generate_password_hash y se comparan con check_password_hash. Flask-Login se encarga de las sesiones. Al registrar un usuario sale el mensaje de registro exitoso. Al iniciar sesión bien, te manda al perfil y sale el mensaje de inicio de sesión exitoso. Si la contraseña está mal, sale credenciales inválidas.

Sistema de registro

- Debes iniciar sesión para acceder a esta página.

Iniciar sesión

Usuario

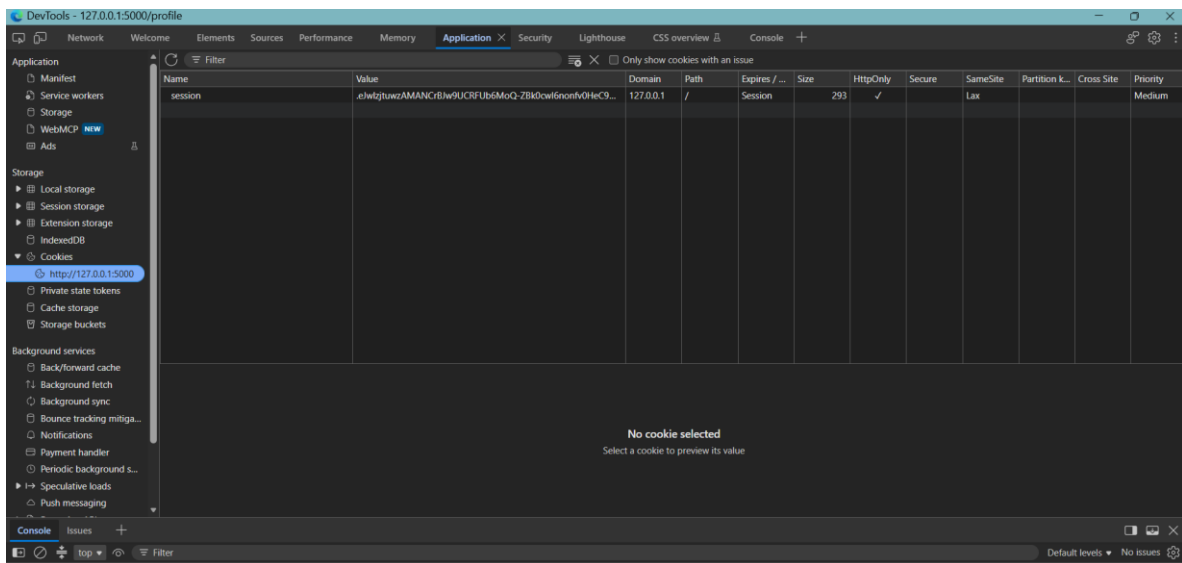
Contraseña

☐ Recordarme

Iniciar sesión

¿No tienes cuenta? [Regístrate](#)

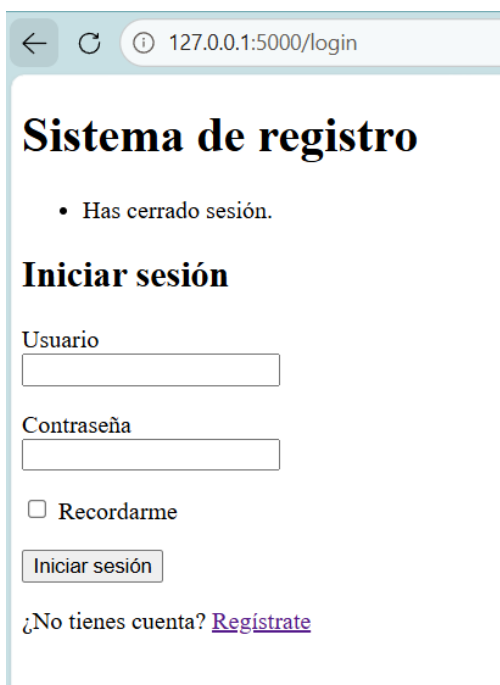
La ruta del perfil está protegida con el decorador `@login_required`. Si intentas entrar sin haber iniciado sesión, Flask-Login te manda al login y te avisa con un mensaje.



Se configuraron las cookies con HttpOnly, SameSite y Secure. En las herramientas del navegador se ve que la cookie session tiene la casilla HttpOnly marcada, lo que significa que JavaScript no puede leerla.



Los mensajes flash se muestran desde base.html. También se hicieron páginas propias para los errores 401, 403 y 404.



El usuario puede cerrar sesión y se le muestra un mensaje confirmando que ya salió.

CONCLUSIONES

Con esta práctica entendí cómo funciona CSRF y por qué es importante proteger los formularios con un token. También aprendí que las contraseñas nunca se deben guardar en texto plano, sino con hash. Flask-Login hace más fácil manejar las sesiones y proteger rutas. Las cookies con HttpOnly y SameSite ayudan a que la sesión sea más segura. Lo que más me costó fue entender por qué el navegador seguía mandando el formulario con caché cuando ya había quitado el token. Lo resolví usando el comando curl en la terminal para mandar el POST directo y ver el error 400.

<https://github.com/miguelxbox0531-sys/A1.1-Pr-ctica-guiada-Implementaci-n-de-seguridad-en-Flask.git>